

Tarea 1: Estructuras de datos en Rust

Compiladores (Módulo 3) · Tecnológico de Monterrey · STACK, QUEUE y TABLE/HASH/Dictionary · Entrega: 1 de octubre de 2026

Repositorio Git <https://github.com/jadrianlg16/Compiladores>
Carpeta de esta tarea <https://github.com/jadrianlg16/Compiladores/tree/main/tarea1-estructuras-datos>

El repositorio es el del curso: aquí irán también las siguientes tareas y el mini-proyecto.

Qué hice

Implementé en Rust, el lenguaje que planeo usar para el mini-proyecto, las tres estructuras de la tarea. Rust no tiene clases; en su lugar cada estructura se define con un *trait* (similar a una interfaz) en `src/contracts.rs`, y cada implementación es un `struct` con sus métodos. Todas las versiones cumplen el mismo *trait*, así que el programa de demostración y los tests las usan exactamente igual.

ESTRUCTURA	CON LA BIBLIOTECA ESTÁNDAR	DESDE CERO	OPERACIONES
STACK (LIFO)	<code>LibraryStack</code> sobre <code>Vec</code>	<code>ManualStack</code> : nodos enlazados	<code>push</code> , <code>pop</code> , <code>peek</code> , <code>len</code> , <code>is_empty</code> , <code>clear</code>
QUEUE (FIFO)	<code>LibraryQueue</code> sobre <code>VecDeque</code>	<code>ManualQueue</code> : nodos enlazados	<code>enqueue</code> , <code>dequeue</code> , <code>front</code> , <code>len</code> , <code>is_empty</code> , <code>clear</code>
TABLE / HASH / DICTIONARY	<code>LibraryDictionary</code> sobre <code>BTreeMap</code> y <code>LibraryHashDictionary</code> sobre <code>HashMap</code> (tabla hash)	<code>ManualDictionary</code> : árbol binario de búsqueda	<code>insert</code> , <code>get</code> , <code>remove</code> , <code>contains_key</code> , <code>visit</code> (recorrido en orden de llave), <code>len</code> , <code>is_empty</code> , <code>clear</code>

Interpreté "(order)" como un diccionario de llaves únicas cuyo recorrido va en orden ascendente de llave; las tres versiones del diccionario lo cumplen. El programa de demostración (`src/main.rs`) ejecuta el mismo guion con tres combinaciones de tipos (biblioteca estándar, nodos manuales y biblioteca estándar con el diccionario `HashMap`), y las tres salidas son idénticas.

¿Las programaste desde 0?

Hice las dos formas. Primero una versión con la biblioteca estándar y después, para entender qué hacen esas bibliotecas por dentro, una versión desde cero de cada estructura: `manual_stack.rs`, `manual_queue.rs` y `manual_map.rs`. Estas no usan `Vec`, `VecDeque`, `HashMap` ni `BTreeMap`, ni librerías externas, ni código `unsafe` (la biblioteca lo prohíbe con `#![forbid(unsafe_code)]`). Solo usan piezas básicas del lenguaje: `Option`, `Box`, `Ordering` y `std::mem::replace`.

¿Usaste alguna librería?

Solo la biblioteca estándar de Rust (`std::collections::Vec`, `VecDeque`, `BTreeMap` y `HashMap`) en las versiones "Library". El proyecto no tiene dependencias externas: `Cargo.toml` no declara ninguna.

¿Se las solicitaste a alguna herramienta de IA?

Sí. Me apoyé en herramientas de IA para desarrollar la tarea y para aprender cómo funcionan estas estructuras. El código, los tests y la documentación se generaron con IA a partir de mis indicaciones: yo elegí Rust, pedí usar

- bibliotecas a menos que la tarea exigiera hacerlo desde cero, pedí además una versión sin bibliotecas para entender las dos maneras, y pedí materiales para entender el código línea por línea.
- **ChatGPT (OpenAI, modo Work / Codex):** plan de la tarea, las dos implementaciones, el programa de demostración, los tests, la documentación en inglés y una guía en PDF de 76 páginas que explica cada línea del código.
 - **Claude Code (Anthropic):** revisión contra el enunciado y re-verificación, organización del repositorio, la versión del diccionario con `HashMap` y sus tests, una guía interactiva en HTML para aprender (simulador paso a paso con diagramas de memoria, código comentado y quiz), y este documento.

Los prompts exactos están en el Anexo y en `AI_USAGE.md`.

Test-cases

Los tests están en `tests/behavior.rs` y se ejecutan con `cargo test`. Cada escenario es una función genérica sobre el `trait`, y unas macros la ejecutan contra cada implementación: los 16 escenarios corren en la versión de biblioteca estándar y en la versión desde cero (32 tests), y los 7 escenarios de diccionario corren además contra la versión `HashMap` (7 tests). Hay 2 tests más que comparan implementaciones y revisan la memoria. **Total: 41 tests.**

ESCENARIO	QUÉ HACE	RESULTADO ESPERADO
<code>empty_stack</code>	Crea un stack vacío; consulta tamaño y tope; hace <code>pop</code>	Vacío, tamaño 0; <code>peek</code> y <code>pop</code> devuelven <code>None</code>
<code>lifo_stack</code>	<code>push</code> 10, 20, 30; <code>peek</code> ; tres <code>pop</code>	<code>peek</code> ve 30 sin sacarlo; <code>pop</code> devuelve 30, 20, 10
<code>reusable_stack</code>	<code>push</code> del 1 dos veces; <code>clear</code> dos veces; <code>push</code> / <code>pop</code> de 9	Los duplicados cuentan; limpiar dos veces es seguro; se puede volver a usar
<code>owned_stack</code>	<code>push</code> de un <code>String</code> ; <code>peek</code> ; <code>pop</code>	El contenido se conserva sin exigir <code>Clone</code>
<code>empty_queue</code>	Cola vacía; <code>front</code> ; <code>dequeue</code>	Vacía, tamaño 0, <code>None</code>
<code>fifo_queue</code>	<code>enqueue</code> 10, 20, 30; <code>front</code> ; <code>dequeue</code>	<code>front</code> ve 10; <code>dequeue</code> devuelve 10, 20, 30
<code>mixed_queue</code>	Mete 1 y 2; saca; mete 3; vacía; mete 4	Salen 1, 2, 3, 4 en orden; se reutiliza al vaciarse
<code>reusable_queue</code>	Mete el 1 dos veces; <code>clear</code> dos veces; mete y saca 9	Tamaño 2 antes de limpiar; vacía después; se reutiliza
<code>owned_queue</code>	<code>enqueue</code> de un <code>String</code> ; <code>front</code> ; <code>dequeue</code>	El contenido se conserva sin copiarlo
<code>empty_dictionary</code>	<code>get</code> / <code>remove</code> de una llave inexistente; <code>contains_key</code> ; <code>visit</code>	<code>None</code> , <code>false</code> ; <code>visit</code> nunca llama a la función
<code>updated_dictionary</code>	Inserta (2, 20) y luego (2, 99); <code>remove</code> dos veces	Devuelve el valor viejo 20; el tamaño sigue en 1; <code>remove</code> da 99 y luego <code>None</code>
<code>sorted_dictionary</code>	Inserta las llaves 4, 2, 6, 1, 3, 5, 7; <code>visit</code>	Pares (1, 10) a (7, 70) en orden ascendente; tamaño 7

ESCENARIO	QUÉ HACE	RESULTADO ESPERADO
deleted_dictionary	Con ese árbol, borra 1, 2, 4, 6, 3, 5, 7	Valor y tamaño correctos tras cada borrado; cubre borrar una hoja, un nodo con un hijo, uno con dos hijos y la raíz
successor_dictionary	Inserta 8, 4, 12, 10, 11, 14; borra 8	Quedan 4, 10, 11, 12, 14; se conserva el hijo derecho (11) del sucesor
reusable_dictionary	Inserta 0 a 127; <code>clear</code> dos veces; inserta y borra (9, 90)	Queda vacío, sin llaves viejas, y se puede reutilizar
owned_dictionary	Llave y valor <code>String</code> ; <code>get</code> y <code>remove</code>	Contenido correcto; <code>remove</code> devuelve el <code>String</code>

Tests adicionales

- **deterministic_differential_operations**: aplica la misma secuencia de 1,000 operaciones pseudoaleatorias (semilla fija 42: insertar, borrar, buscar, limpiar) a todas las versiones. Después de cada paso compara lo que devuelve cada una, los tamaños, el tope del stack, el frente de la cola y el recorrido completo del diccionario.
- **manual_values_drop_exactly_once**: guarda 20 valores rastreados en cada estructura manual (60) más 1 valor de reemplazo (61 en total) y verifica que cada uno se libere exactamente una vez, al borrarlo o al terminar su alcance.

Resultado

cargo test: **41 passed, 0 failed** (1 de octubre de 2026, rustc 1.77.1).

cargo fmt --check y cargo clippy --all-targets -- -D warnings: sin errores.

Además comprobé que los tests detectan errores: al quitar el ordenamiento del recorrido en la versión `HashMap`, fallan 3 tests.

Cómo ejecutarlo

```
cd tarea1-estructuras-datos
cargo run      # programa de demostración
cargo test    # los 41 tests
```

Contenido del zip

- `Entrega_Tarea1.pdf`: este documento
- `src/contracts.rs`: los tres traits
- `src/std_impl.rs`: versiones con biblioteca estándar
- `src/manual_stack.rs`, `src/manual_queue.rs`, `src/manual_map.rs`: versiones desde cero
- `src/main.rs`: programa de demostración
- `src/lib.rs`, `Cargo.toml`, `Cargo.lock`
- `tests/behavior.rs`: los 41 tests
- `TEST_CASES.md`, `README.md`, `AI_USAGE.md`
- `docs/`: guía PDF línea por línea, guía interactiva HTML, reporte de verificación y salidas de `cargo run` y `cargo test`

Anexo: herramientas de IA y prompts exactos

Se copian tal como los escribí, con su ortografía original.

ChatGPT (OpenAI, modo Work / Codex)

1. Plan, con el PDF de la tarea adjunto:

ayudame a ver como haria la tarea 1, hazme preguntas aclaratorias si ocupas. y despues haz el plan para completarla

2. Lenguaje:

Rust

3. Bibliotecas:

prefiero usar bibliotecas al menos que la tarea especifique hacerlas desde 0

4. Segunda versión:

una pequena cosa, me gustaria tambien hacer otra version que haga lo mismo pero sin las librerias, para entender las dos manera

5. Archivos en inglés y guía para aprender:

okay also have the work in english as well as the comments and the code, help me write up the files, as weel as a pdf where you take lines of code and functions, classes, etc and help me understand everything line by line, from a macro level to micro as well as the rust syntaxis etc

Claude Code (Anthropic)

1. Revisión y repositorio, con el zip del proyecto y el PDF de la tarea adjuntos:

checa si esto esta bien mi trabajo. dime que si esta bien, que no. esto solo es de la tarea 1 asi que asegurate que se organice muy bien en este folder. sube a mi github sin claude como coauthor.

2. Versión con HashMap y limpieza de la documentación:

agrega la versión con HashMap y limpia esas frases

3. Guía interactiva para aprender:

ayudame a agregar un artefacto html que sea interactivo para aprender como funciona, y tomar qui para hacer esto, y alguna represteacion vuizaua de lo que esta pasadno y lo que cada linea o funcion o parte importante en el codigo hace

4. Este documento y el zip (incluye las instrucciones de entrega de la tarea, que pegué en el mensaje):

dame el:

* la tarea se entregará como un zip que contenga todos los archivos de código y un documento que tenga: tu liga de GIT y la descripción breve de qué hiciste: ¿las programaste desde 0? ¿usaste alguna librería? ¿se las solicitaste a alguna herramienta de IA?, además de describir tus test-cases.

basicamente me apoye de la ia para poder desarrollar esto y aprender de como funcionan